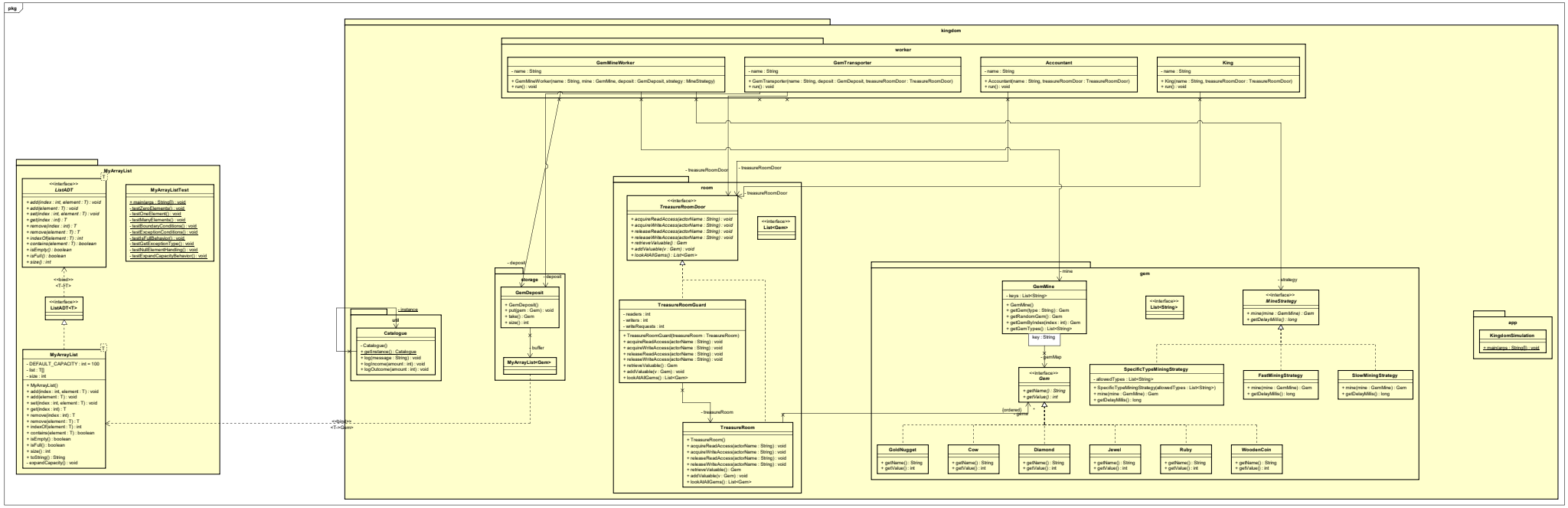
pkg
kingdom
worker
GemMineWorker
- name : String
+ GemMineWorker(name : String, mine : GemMine, deposit : GemDeposit, strategy : MineStrategy)
+ run() : void
GemTransporter
- name : String
+ GemTransporter(name : String, deposit : GemDeposit, treasureRoomDoor : TreasureRoomDoor)
+ run() : void
Accountant
- name : String
+ Accountant(name : String, treasureRoomDoor : TreasureRoomDoor)
+ run() : void
King
- name : String
+ King(name : String, treasureRoomDoor : TreasureRoomDoor)
+ run() : void
MyArrayList
<<interface>>
ListADT
+ add(index : int, element : T) : void
+ add(element : T) : void
+ set(index : int, element : T) : void
+ get(index : int) : T
+ remove(index : int) : T
+ remove(element : T) : T
+ indexOf(element : T) : int
+ contains(element : T) : boolean
+ isEmpty() : boolean
+ isFull() : boolean
+ size() : int
MyArrayListTest
+ main(args : String[]) : void
- testZeroElements() : void
- testOneElement() : void
- testManyElements() : void
- testBoundaryConditions() : void
- testExceptionConditions() : void
- testIsFullBehavior() : void
- testGetExceptionType() : void
- testNullElementHandling() : void
- testExpandCapacityBehavior() : void
<<bind>>
<T->T>
<<interface>>
ListADT<T>
MyArrayList
- DEFAULT_CAPACITY : int = 100
- list : T[]
- size : int
+ MyArrayList()
+ add(index : int, element : T) : void
+ add(element : T) : void
+ set(index : int, element : T) : void
+ get(index : int) : T
+ remove(index : int) : T
+ remove(element : T) : T
+ indexOf(element : T) : int
+ contains(element : T) : boolean
+ isEmpty() : boolean
+ isFull() : boolean
+ size() : int
+ toString() : String
- expandCapacity() : void
<<bind>>
<T->Gem>
- instance
util
Catalogue
- Catalogue()
+ getInstance() : Catalogue
+ log(message : String) : void
+ logIncome(amount : int) : void
+ logOutcome(amount : int) : void
- deposit
storage
- deposit
GemDeposit
+ GemDeposit()
+ put(gem : Gem) : void
+ take() : Gem
+ size() : int
- buffer
MyArrayList<Gem>
- treasureRoomDoor
- treasureRoomDoor
room
<<interface>>
TreasureRoomDoor
+ acquireReadAccess(actorName : String) : void
+ acquireWriteAccess(actorName : String) : void
+ releaseReadAccess(actorName : String) : void
+ releaseWriteAccess(actorName : String) : void
+ retrieveValuable() : Gem
+ addValuable(v : Gem) : void
+ lookAtAllGems() : List<Gem>
<<interface>>
List<Gem>
TreasureRoomGuard
- readers : int
- writers : int
- writeRequests : int
+ TreasureRoomGuard(treasureRoom : TreasureRoom)
+ acquireReadAccess(actorName : String) : void
+ acquireWriteAccess(actorName : String) : void
+ releaseReadAccess(actorName : String) : void
+ releaseWriteAccess(actorName : String) : void
+ retrieveValuable() : Gem
+ addValuable(v : Gem) : void
+ lookAtAllGems() : List<Gem>
- treasureRoom
TreasureRoom
+ TreasureRoom()
+ acquireReadAccess(actorName : String) : void
+ acquireWriteAccess(actorName : String) : void
+ releaseReadAccess(actorName : String) : void
+ releaseWriteAccess(actorName : String) : void
+ retrieveValuable() : Gem
+ addValuable(v : Gem) : void
+ lookAtAllGems() : List<Gem>
- mine
gem
GemMine
- keys : List<String>
+ GemMine()
+ getGem(type : String) : Gem
+ getRandomGem() : Gem
+ getGemByIndex(index : int) : Gem
+ getGemTypes() : List<String>
key : String
- gemMap
<<interface>>
Gem
+ getName() : String
+ getValue() : int
<<interface>>
List<String>
- strategy
<<interface>>
MineStrategy
+ mine(mine : GemMine) : Gem
+ getDelayMillis() : long
SpecificTypeMiningStrategy
- allowedTypes : List<String>
+ SpecificTypeMiningStrategy(allowedTypes : List<String>)
+ mine(mine : GemMine) : Gem
+ getDelayMillis() : long
FastMiningStrategy
+ mine(mine : GemMine) : Gem
+ getDelayMillis() : long
SlowMiningStrategy
+ mine(mine : GemMine) : Gem
+ getDelayMillis() : long
(ordered)
gems
GoldNugget
+ getName() : String
+ getValue() : int
Cow
+ getName() : String
+ getValue() : int
Diamond
+ getName() : String
+ getValue() : int
Jewel
+ getName() : String
+ getValue() : int
Ruby
+ getName() : String
+ getValue() : int
WoodenCoin
+ getName() : String
+ getValue() : int
app
KingdomSimulation
+ main(args : String[]) : void